Relatório do experimento de vazamento

Autor: Mauricio Matias **Data:** 30/10/2026 **Disciplina:** Mineração de Dados 2026/2
Repositório: https://github.com/mauriciomatias-stack/lab-etl-varejo-mercatto

1. O experimento

Pergunta: dado o comportamento do cliente até `2025-06-30`, ele compra de novo entre `2025-07-01` e `2025-09-28`? Duas tabelas, geradas pelo mesmo SQL (`etl/04_abt.py`), com os mesmos 22.106 clientes e o mesmo alvo (4.801 recompram, taxa 0,217). A única diferença é o cálculo de três atributos. No Orange, o mesmo fluxo para as duas: File, Data Sampler (70% treino, 30% teste, estratificado), Naive Bayes, Regressão Logística e Árvore, Test and Score em "Test on test data" (classe-alvo 1), Matriz de Confusão e ROC.

2. Os três vazamentos

#	Atributo	Como foi feito errado	Por que é vazamento
1	`recencia_dias`	filtro `data_pedido <= corte` trocado por TRUE	a "última compra" pode estar na janela do alvo; quem recomprou tem recência pequena ou negativa
2	`frequencia_pedidos`	mesmo filtro removido	conta os pedidos da própria janela do alvo; quem recomprou tem um pedido a mais
3	`ticket_medio`	z-score com média e desvio de todas as linhas	a escala é calculada antes da partição treino/teste, então o teste influencia o treino

Os dois primeiros vazam o **alvo** para dentro dos atributos. O terceiro vaza a **distribuição do teste** para dentro do pré-processamento.

3. Resultado lado a lado (30% de teste, classe 1)

Modelo	Versão	AUC	CA	F1	Prec	Recall
Naive Bayes	correta	0,552	0,767	0,137	0,353	0,085
Naive Bayes	vazamento	0,871	0,796	0,574	0,526	0,633
Regressão Logística	correta	0,549	0,783	0,000	0,000	0,000
Regressão Logística	vazamento	0,875	0,808	0,466	0,590	0,385
Árvore	correta	0,507	0,716	0,186	0,246	0,150
Árvore	vazamento	0,954	0,964	0,915	0,931	0,900

Matrizes de confusão da versão com vazamento (teste de 6.631 clientes; verdadeiro negativo, falso positivo, falso negativo, verdadeiro positivo): Naive Bayes 4.369 / 822 / 529 / 911; Árvore 5.095 / 96 / 144 / 1.296. Na versão correta o Naive Bayes acerta só 8,5% dos que recompram.

O que muda. O AUC do Naive Bayes sobe de 0,552 para 0,871 e o da Árvore de 0,507 para 0,954. A Árvore é a que mais ganha porque consegue cortar exatamente no limite que o vazamento cria (recência negativa ou frequência acima do que o cliente tinha no corte). A Regressão Logística, que só traça uma fronteira linear, ganha menos.

Os números da versão correta ficam abaixo da faixa que o professor indica (0,62 a 0,72). Antes de aceitar, testei a ABT: o AUC de cada atributo isolado fica entre 0,48 e 0,54; frequência nos 90, 180 e 365 dias antes do corte dá 0,51 a 0,53; o volume de pedidos por trimestre é constante (cerca de 6.800). Os dados quase não têm sinal de recompra. Não mexi no alvo para chegar à faixa. Esta ABT não prediz bem; isso é um resultado, não um erro de pipeline.

4. Como descobrir o vazamento se só o segundo resultado chegasse

Recebendo apenas a tabela da versão com vazamento (AUC 0,95 para um problema de recompra), as pistas, em ordem de custo:

1. **O próprio número.** AUC acima de 0,93 para prever comportamento de cliente com atributos de cadastro e histórico é raro. O resultado bom demais é o primeiro alarme. Um AUC de 0,55 sem vazamento é mais plausível para estes dados.
2. **Olhar o dado, não o modelo.** `recencia_dias` tem valor mínimo −183, e 8.615 clientes (38,97%) têm recência negativa: uma "última compra" depois do corte é impossível. Isso se vê em um `min()` ou em um histograma.
3. **AUC de cada atributo sozinho.** `recencia_dias` passa de 0,478 para 0,192 (0,808 invertido) e `frequencia_pedidos` de 0,530 para 0,712. Um atributo isolado que prevê o futuro quase tão bem quanto o modelo inteiro denuncia o vazamento.
4. **Média por classe.** A frequência média de quem recompra vai a 4,29 contra 2,90 de quem não recompra; na versão correta, 2,86 e 2,65. A diferença some quando se respeita o corte.

5. **Recalcular por fora.** A asserção `verifica_sem_vazamento` recalcula frequência, valor monetário e recência em pandas, com o filtro de data explícito, e compara linha a linha com a ABT. Na versão com vazamento ela falha 3 vezes (8.615 linhas); na correta, 0.
6. **O z-score não é pego por nenhuma dessas pistas.** O atributo continua com o mesmo AUC isolado (0,503). Só se encontra pela revisão do código: toda normalização, imputação ou seleção de atributos precisa ser ajustada no treino e aplicada ao teste. Aqui o efeito é pequeno e não o medi separadamente; o perigo está justamente em passar despercebido.

5. A acurácia engana

Na versão correta, a Regressão Logística tem CA 0,783 e F1 0,000: ela prevê "não recompra" para todo mundo. A proporção da classe majoritária é 0,783 (1 − 0,217), então a acurácia é a de quem nunca prevê recompra. Com 78% de CA a máquina não aprendeu nada. Por isso o experimento relata AUC, F1, precisão e recall da classe 1 e as matrizes de confusão. Na versão com vazamento, a CA do Naive Bayes (0,796) é só 1,3 ponto acima disso, mas o F1 vai de 0,137 para 0,574: a acurácia quase não mostra a diferença, a métrica da classe 1 mostra.

6. Conclusão e limites

- O vazamento cabe em uma linha de SQL e multiplica o AUC (de 0,51 para 0,95 na Árvore). Um resultado excelente numa tabela-base é suspeito até ser provado.
- A prova é uma asserção que recalcula o atributo com o corte explícito, mais a leitura do dado (valores impossíveis) e o AUC de cada atributo.
- Limites: o status do pedido é o da extração, sem data de cancelamento, então a ABT correta tem um pequeno resíduo de informação posterior ao corte (RDT-14). Uma só partição 70/30 foi usada; os valores do Orange variam um pouco com a semente. O ganho do z-score não foi isolado.
- Reprodução: `04_abt.py --correta`, `04_abt.py --vazamento`, `05_exportar_orange.py`, fluxos em `orange/fluxo_correta.ows` e `orange/fluxo_vazamento.ows`.